

基于残差多树遗传编程的蚁群优化状态转移与信息素更新规则联合设计

第一作者, 第二作者, 第三作者

摘要—蚁群优化的性能主要取决于状态转移规则和信息素更新规则。这两类规则通常由研究者手工设计。遗传编程可以自动生成规则。但是, 直接替换完整规则可能破坏原算法中有效的结构。本文提出残差多树遗传编程蚁群优化方法, 简称 RMTGP-ACO。该方法使用两棵强类型树。第一棵树对状态转移分数进行有界修正。第二棵树在固定预算内重新分配信息素增量。当两棵树都输出零时, 算法严格退化为原始 ACO。本文在 AS、ACS 和 MMAS 上实现该方法。训练只使用均匀分布的 TSP100 实例。每个 ACO 变体进行三个独立 GP 运行。选出的程序在 TSP50、TSP100、TSP500 和 TSP1000 上测试。在当前 12 个“算法变体-规模”组合中, Full-F1 的平均参考差距均低于原始 ACO。下降幅度为 0.0736 至 7.8461 个百分点。融合 CUDA 评估器在登记的基准上取得了相对 16 线程 CPU 的 14.626 倍单 GPU 加速。这些结果属于初步结果。残差接口、双树结构、终端集合和函数集合的消融实验仍需完成。

关键词—蚁群优化, 遗传编程, 超启发式, 多树遗传编程, 残差学习, 旅行商问题。

I. 引言与研究动机

蚁群优化 (ant colony optimization, ACO) 通过信息素和启发式信息构造解 [1], [2]。状态转移规则决定下一步选择。信息素更新规则决定后续搜索分布。这两类规则都很重要。现有算法通常使用固定公式。

遗传编程 (genetic programming, GP) 可以进化可执行表达式 [3], [4]。上一篇 GP-ACO 工作直接进化完整的状态转移规则 [5]。该方法证明了自动设计规则的可行性。但是, 完整替换会删除原 ACO 的分数结构。进化表达式也可能产生过大的数值。此外, 状态转移和信息素更新之间存在反馈关系。只学习其中一个部分可能限制搜索能力。

本文研究一个简单问题: 能否保留原 ACO 的主体, 只让 GP 学习有限的修正? 为此, 本文提出 RMTGP-ACO。方法包含一棵状态转移树和一棵信息素树。两棵树使用不同的强类型。它们共享同一个适应度和同一个总节点预算。

本文的技术贡献如下。

- 提出有界的状态转移残差接口。零输出恢复原始状态转移规则。
- 提出预算守恒的信息素残差接口。它只重新分配原算法的增量。
- 提出强类型双树表示。两棵树共享 31 个有效节点。
- 提出面向 GP-ACO 嵌套评估的融合 GPU 架构。

当前实验是三个独立 GP 运行的先导实验。它可以回答完整方法是否优于原始 ACO。它还不能确定改进来自哪一个设计。本文对未完成的问题明确标记为“待完成”。

II. 问题定义与基线算法

A. 欧氏旅行商问题

设城市集合为 $V = \{1, \dots, n\}$ 。城市 i 的二维坐标为 c_i 。路线 π 是全部城市的一个排列。城市距离和闭合路线长度

定义为

$$d_{ij} = \|c_i - c_j\|_2, \quad L(\pi) = \sum_{q=1}^n d_{\pi_q, \pi_{q+1}}, \quad \pi_{n+1} = \pi_1. \quad (1)$$

优化目标是找到长度最小的路线。

每个数据实例都带有一条参考路线。本文使用 float64 重新计算参考长度。数据文件没有为每条路线提供全局最优性证书。因此, 本文使用“参考长度”, 而不使用“已知最优长度”。设实例 i 的参考长度为 L_i^{ref} 。算法所得长度为 L_i 。参考差距定义为

$$g_i = 100 \frac{L_i - L_i^{\text{ref}}}{L_i^{\text{ref}}} (\%). \quad (2)$$

该指标越小越好。例如, 参考长度为 100, 算法长度为 102, 则参考差距为 2%。由于参考路线不一定被证明为全局最优, 参考差距在理论上可以小于零。

B. 原始 ACO 规则

边 (i, j) 的信息素为 τ_{ij} 。启发式值为 $\eta_{ij} = 1/(d_{ij} + \epsilon_d)$ 。原始转移分数为 $s_{aij}^0 = \tau_{ij}^\alpha \eta_{ij}^\beta$ 。AS 和 MMAS 按归一化分数采样。ACS 以概率 q_0 选择最高分城市, 否则按分数采样。三个算法都使用候选列表。候选列表没有可行城市时, 算法检查全部未访问城市。

AS 对全部信息素挥发, 并使用所有蚂蚁的路线进行增强。ACS 在构造过程中执行局部更新, 并在每轮结束后增强全局最优路线。MMAS 按原有调度选择增强路线。MMAS 还使用动态上下界和重启机制。本文实现遵循 ACOTSP 1.03 的算法语义 [6]。三个算法均使用 32 只蚂蚁和 500 次 ACO 迭代。局部搜索关闭。

C. 上一篇 GP-ACO 基线

上一篇工作使用一棵 GP 树替换完整的状态转移表达式 [5]。本文将其匹配实现记为 Legacy-GP。Legacy-GP 使用原论文的终端和函数结构。它只修改状态转移, 不修改信息素更新。本文会在当前数据、ACO 预算和随机数计划下重新训练该方法。不同计算预算下的已发表数字不会直接放入配对比较。

III. RMTGP-ACO 方法

A. 总体结构

一个个体包含两棵树, 即状态转移树 T_τ 和信息素树 T_ϕ 。图 1 给出整体流程。状态转移树在路线构造时运行。信息素树在当前轮的路线全部生成后运行。AS、ACS 和 MMAS 的原有控制逻辑保持不变。

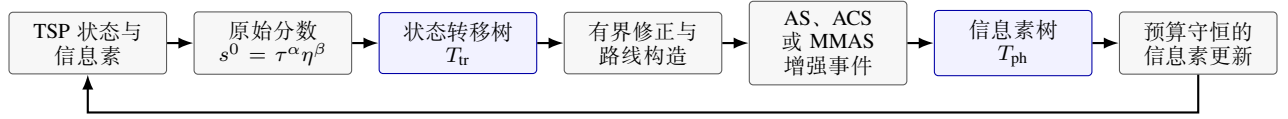


图 1. RMTGP-ACO 的数据流。蓝色模块由 GP 进化。其余模块保留对应 ACO 变体的语义。

B. 强类型双树表示

本文使用 DEAP 的强类型 GP [7], [8]。表 I 给出两个类型。TrField 与 PhField 是不同的 Python 标记类。因此，DEAP 不允许两类树之间交换子树。这一限制避免了形状正确但语义错误的表达式。

两棵树共享一个适应度。它们也共享总节点预算：

$$N_{tr} + N_{ph} \leq 31. \quad (3)$$

零树不计算有效节点。任一非零树的深度不超过 5。该设计保证双树方法与单树方法具有相同的总表达能力上限。它不是“每棵树 31 个节点”。

C. 状态转移残差

状态转移树不重新生成完整分数。它只对原始分数 s_{aij}^0 进行有限修正：

$$\tilde{s}_{aij} = s_{aij}^0 \left[1 + \frac{1}{3} \tanh(T_{tr}(x_{aij})) \right]. \quad (4)$$

x_{aij} 表示状态转移终端。方括号中的乘数始终位于 2/3 与 4/3 之间。因此，修正后的分数保持为正。当树输出零时，乘数为 1，原始 ACO 被严格恢复。ACO 仍使用自己的采样方式对修正分数进行归一化。

D. 信息素残差

一次信息素增强称为一个事件。AS 每只蚂蚁产生一个事件。ACS 产生一个全局最优事件。MMAS 按原调度产生一个事件。设事件 r 对边 e 的原始增量为 $D_{r,e}^0$ 。信息素树按以下三步重新分配增量：

$$\begin{aligned} h_{r,e} &= 1 + \frac{1}{3} \tanh(T_{ph}(z_{r,e})), \\ w_{r,e} &= D_{r,e}^0 h_{r,e}, \\ \tilde{D}_{r,e} &= B_r \frac{w_{r,e}}{\sum_{f \in E(\pi_r)} w_{r,f}}, \quad B_r = \sum_{f \in E(\pi_r)} D_{r,f}^0. \end{aligned} \quad (5)$$

$z_{r,e}$ 表示信息素终端。第一步产生有界权重。第二步调整各条边的相对份额。第三步把总量缩放回原始预算 B_r 。因此，增强边集合不变，总增量也不变。零输出严格恢复原始更新。该残差不会改变 ACS 局部更新、MMAS 上下界或 MMAS 重启逻辑。

E. 状态转移终端集合

所有连续终端都转为有界的相对值。标准化的一般做法很简单。先在当前有效维度计算均值和标准差。再计算 z-score。最后使用 $\tanh$ 将值限制到 $[-1, 1]$ 。掩码位置设为零。

表 II 给出全部状态转移终端。Core 使用前四个终端。Full 使用全部八个终端。BaseConf 中的 p^0 是原始 ACO 的归一化概率。 k 是当前可行城市数。

F. 信息素终端集合

表 III 给出信息素终端。每个值都对应一个增强事件中的一条路线边。Core 使用 EdgeEta、EdgeTau、NNRank 和 SourceQuality。Full 使用全部七个终端。

两个终端集合都包含常数 $-1, -0.5, 0, 0.5, 1$ 。它们还包含一个从 $U[-1, 1]$ 采样的随机常数。随机常数变异加入标准差为 0.1 的高斯噪声。变异后的值截断到 $[-1, 1]$ 。

G. 函数集合与数值保护

表 IV 给出两个函数配置。所有运算都逐元素执行。F1 在 F0 上增加 MIN、MAX 和 ABS。

保护除法定义为 $PDIV(x, y) = xy/(y^2 + 10^{-6})$ 。它在 $y = 0$ 附近连续。每次函数计算后，NaN 被替换为零。正负无穷被替换为 10 和 -10。其余结果也截断到 $[-10, 10]$ 。树根外部还会应用一次 $\tanh$ 。

Legacy-GP 使用原研究的保护除法。当分母绝对值不大于 10^{-6} 时，它返回 1。Matched-Replace 使用 Full 终端和 F1 函数。它将树输出经过截断和 softplus 后直接作为完整转移分数。

H. 进化过程与适应度

联合种群包含四种初始化模式。其中 10% 为双零树。20% 只启用状态转移树。20% 只启用信息素树。其余 50% 同时启用两棵树。非零树使用深度 2 至 4 的 ramped half-and-half 初始化。

每代保留 10 个精英。其余个体使用交叉、变异和复制。三者概率分别为 0.80、0.15 和 0.05。锦标赛大小为 4。交叉只在同一强类型内执行。变异包括子树变异、节点替换和随机常数扰动。三者变异内部的概率为 0.50、0.30 和 0.20。违反深度或节点限制的后代由有效亲本替代。

设训练实例集合为 $\mathcal{I}$ ，ACO 随机种子集合为 $\mathcal{S}$ 。个体 θ 的适应度为所有参考差距的平均值：

$$F(\theta) = \frac{1}{|\mathcal{I}||\mathcal{S}|} \sum_{i \in \mathcal{I}} \sum_{s \in \mathcal{S}} g_{i,s}(\theta). \quad (6)$$

进化直接最小化 F 。适应度不是“相对 baseline ACO 的提升”。这种定义使不同 ACO 变体使用同一个质量尺度。baseline 的差距只用于结果对比。本文记 $\Delta g = g_{\text{method}} - g_{\text{ACO}}$ 。 $\Delta g < 0$ 表示学习方法更好。适应度不含简约惩罚。当性能差异在预设容差内时，程序大小只用于选择较小程序。

结构哈希会跳过同一代内的重复个体。GPU 编译器还会合并行为相同的程序。这些优化不改变适应度定义。

IV. 实验设计

A. 研究问题

表 V 给出本文需要回答的研究问题。RQ1、RQ6 的规模迁移部分和 RQ7 已有先导结果。其余问题需要完成登记的对照实验。

表 I
两棵 GP 树的输入、输出和执行位置

树	强类型与形状	维度含义	输入与输出	执行位置
状态转移树	TrField, [B, M, K]	B 为实例批量, M 为蚂蚁数, K 为当前候选宽度	全部终端和树输出都具有相同形状	每个路线构造步骤
信息素树	PhField, [B, R, n]	R 为增强事件数, n 为路线边数	全部终端和树输出都具有相同形状	每轮 ACO 的增强阶段

表 II
状态转移终端。全部输出广播为 TrField。

终端	计算方法	直观含义	配置
RTau	对当前可行边的 $\log(\max(\tau, \epsilon))$ 做标准化, 再取 $\tanh$	候选边相对于当前可行集合的信息素强弱	Core、Full
REta	对当前可行边的 $\log(\max(\eta, \epsilon))$ 做标准化, 再取 $\tanh$	候选边相对于当前可行集合的距离启发式强弱	Core、Full
BaseConf	计算 $\tanh[\log(p^0 + \epsilon) + \log k]$	原始分布对该候选的相对信心。均匀分布时为零	Core、Full
DistRank	距离名次为 r 时, 计算 $1 - 2(r - 1)/(k - 1)$; $k = 1$ 时设零	最近城市映射为 1, 最远城市映射为 -1	Core、Full
Entropy	计算 $2H(p^0)/\log k - 1$, 并广播到候选; $k = 1$ 时设为 -1	原始转移分布的不确定性	Full
ConstructProg	计算 $2q/\max(n - 1, 1) - 1$	一条路线内部的构造进度	Full
ACOProg	计算 $2(t - 1)/\max(T - 1, 1) - 1$	整个 ACO 运行的迭代进度	Full
Stagnation	计算 $2\min(s_t/T, 1) - 1$	距离上一次全局最优改进的停滞程度	Full

表 III
信息素终端。全部输出为 PhField。

终端	计算方法	直观含义	配置
EdgeEta	比较边 (u, v) 的对数启发式与两个端点邻域的平均值, 再在来源路线内标准化	该边相对于两个端点局部邻域的距离质量	Core、Full
EdgeTau	对来源路线内的 $\log(\max(\tau_{uv}, \epsilon))$ 做标准化	该边相对于同一增强事件中其他边的信息素强弱	Core、Full
NNRank	将 $u \rightarrow v$ 与 $v \rightarrow u$ 的最近邻名次映射到 $[-1, 1]$, 然后取平均	无方向的局部邻近程度	Core、Full
ColonyFreq	边在当前蚁群中出现 c_e 次时, 计算 $2c_e/M - 1$	当前蚁群对该边的共同选择程度	Full
SourceQuality	计算 $\tanh[(\bar{L} - L_r)/(\sigma_L + \epsilon)]$, 并广播到路线边	来源路线相对于当前蚁群的质量	Core、Full
ACOProg	计算 $2(t - 1)/\max(T - 1, 1) - 1$	整个 ACO 运行的迭代进度	Full
Stagnation	计算 $2\min(s_t/T, 1) - 1$	距离上一次全局最优改进的停滞程度	Full

表 IV
GP 函数集合

配置	二元函数	一元函数
F0	ADD, SUB, MUL, PDIV	NEG
F1	F0, MIN, MAX	NEG, ABS

B. 训练、验证与测试数据

训练池包含 1,280,000 个均匀分布的 TSP100 实例。它由 10 个文件组成。每个文件包含 128,000 个实例。每代使用 32 个不同实例。一个 50 代 GP 运行共使用 1,600 个不同训练实例。不同 GP 运行使用独立的固定抽样计划。

验证文件包含 1,280 个 TSP100 实例。每个 GP 运行使用 32 个实例选择候选。另外 32 个不重叠实例用于独立 gate。测试集与训练集、验证集完全分离。参考路线的边不会作为 GP 终端。

C. 比较方法与消融实验

RQ2 的主要差值是 TR-RGP 的参考差距减去 Matched-Replace 的参考差距。两者使用相同终端、函数、单树节点预算、数据和随机计划。它们只改变规则接入方式。因此, 该对照可以回答 residual 是否更好。

RQ3 比较 Full-F1、TR-RGP 和 PH-RGP。三种方法的总有效节点预算都为 31。另一个容量实验比较 31 和 62 两个总预算。RQ4 使用 2×2 因子设计。RQ5 在训练完成后删除一棵树, 或打乱不同 GP 运行产生的树配对。

D. 算法参数

AS 的 GP 根种子为 1001–1003。ACS 的根种子为 2001–2003。MMAS 的根种子为 3001–3003。测试种子从独立根种子 9001 生成。同一配对比较中的方法共享实例和 ACO 种子。

表 V
研究问题、对照方法和当前状态

问题	问题内容	主要比较	状态
RQ1	Full-F1 是否降低原始 ACO 的参考差距?	Full-F1 与 AS、ACS、MMAS baseline 配对比较	已完成
RQ2	残差状态转移是否优于直接替换完整状态转移?	TR-RGP 与 Matched-Replace	待完成
RQ3	相同总节点预算下, 双树是否优于单树?	Full-F1 与独立训练的 TR-RGP、PH-RGP	待完成
RQ4	改进是否依赖 Full 终端或 F1 函数?	Core/Full 与 F0/F1 的 2×2 因子实验	待完成
RQ5	两棵树之间是否存在联合适应?	删除一棵树, 以及跨 GP 运行打乱树配对	待完成
RQ6	TSP100 训练的程序能否迁移到其他规模和分布?	TSP50/500/1000, 以及 cluster、Gaussian、TSPLIB	规模已完成
RQ7	批量融合实现能提供多大加速?	CPU8、CPU16、单 GPU、双 GPU 与并行 campaign	已完成

算法 1 一次 RMTGP-ACO 模拟

Require: TSP 实例、ACO 变体、 T_{tr} 、 T_{ph}

```

1: 初始化信息素和变体专用状态
2: for  $t = 1, \dots, T$  do
3:   for 每个路线构造步骤 do
4:     建立当前候选集合
5:     计算原始分数  $s^0 = \tau^\alpha \eta^\beta$ 
6:     只计算  $T_{tr}$  实际使用的终端
7:     按式 (4) 修正分数
8:     使用原 ACO 变体的选择规则
9:     ACS 在需要时执行局部更新
10:  end for
11:  计算路线长度并更新最优状态
12:  建立 AS、ACS 或 MMAS 增强事件
13:  只计算  $T_{ph}$  实际使用的终端
14:  按式 (5) 重新分配增量
15:  执行原变体的挥发、边界和重启逻辑
16: end for
17: return 最优路线; 在 CPU 上用 float64 重新计算长度

```

算法 2 GP 训练与候选程序选择

Require: 固定的训练与验证计划

```

1: 载入不可变的 baseline ACO 缓存
2: 初始化 100 个强类型个体
3: for  $h = 1, \dots, 50$  do
4:   载入计划中的 32 个 TSP100 训练实例
5:   全部个体共享同一 ACO 种子
6:   评估唯一程序并计算式 (6)
7:   每 5 代保存前 5 个程序
8:   使用强类型算子生成下一代
9: end for
10: 在 32 个选择实例上用一个种子筛选检查点
11: 对前 5 个候选使用三个配对种子重新评估
12: 在最优值 0.01 个百分点内选择更小程序
13: 在另外 32 个 gate 实例上独立审计
14: return 每个 GP 运行的一个入选程序及一个部署策略

```

E. 候选选择与统计方法

每一代只使用一个 ACO 种子。同一代的所有个体共享该种子和同一批 32 个实例。这种配对可以降低个体之间的随机噪声。baseline ACO 结果提前计算, 并保存在不可变缓存中。

验证先用一个种子筛选检查点。再用三个种子评估最好的五个候选。在最优平均差距的 0.01 个百分点以内, 选

表 VI
TSP100 先导实验的数据配置

用途	数据分区	实例数
训练池	TSP100 uniform	1,280,000
每个 GP 运行	32×50 TSP100	1,600
验证	selection / gate	32/32
测试	TSP50 uniform	1,280
测试	TSP100 uniform	1,280
测试	TSP500 uniform	128
测试	TSP1000 uniform	128
待测	TSP500 cluster / Gaussian	128/128
待测	TSPLIB, $n \leq 500$	42

择节点更少的程序。只有该程序进入独立 gate。若其单侧 95% 上界不超过 0.10 个百分点, 则允许部署。

论文分析保留每个 GP 运行选出的一个程序。三个 GP 运行全部保留。本文不从三个 GP 种子中只选表现最好的一个。gate 部署结果是另一个操作性指标。程序未通过 gate 时, 部署策略回退到 baseline ACO。

主要指标是参考差距。此外记录最差 10% 条件风险、anytime gap 曲线下面积、最优解出现迭代、运行时间和胜/平/负比例。三个 ACO 种子先在实例内求平均。主要统计块为“GP 运行 $\times$ 实例”。本文使用配对 Wilcoxon 符号秩检验 [9]。同一 ACO 变体和 RQ 家族内使用 Holm 校正 [10]。分层 bootstrap 依次重采样 GP 运行、实例和 ACO 种子 [11]。每个区间使用 10,000 次重采样。

当前只有三个独立 GP 运行。因此, 显著性检验和区间只作描述性分析。正式确认实验计划使用 30 个独立 GP 运行。

V. 工程实现与加速

A. 批量评估架构

一个种群有 100 个个体。每个个体在 32 个实例上运行 ACO。每个 ACO 又包含 32 只蚂蚁和 500 次迭代。若在 Python 中逐层循环, 计算成本很高。

实现使用 NumPy、Numba、PyTorch、DEAP 和 CuPy [8], [12]–[15]。CPU 后端建立“个体 $\times$ 实例”的任务矩阵。Numba 编译 ACO 内层循环。GPU 后端把强类型树编译为紧凑的 postfix 程序。它先删除结构重复和行为重复的程序。然后建立“程序 $\times$ 实例”的任务网格。

一个 CUDA block 负责一个“程序–实例”任务。32 个线程与 32 只蚂蚁对应。该 block 完成全部 500 次 ACO 迭代。两棵 GP 树也在同一个 kernel 中执行。问题张量和不依赖树的特征常驻显存。计数器随机数生成器保证任务切分后仍可复现。

表 VII
比较方法、受控差异和实验用途

方法	状态转移	信息素更新	终端/函数	用途	状态
Baseline ACO	不修改	不修改	无	原始 AS、ACS 或 MMAS	已完成
Legacy-GP	完整替换	不修改	上一研究配置	在当前预算下复现上一篇 GP-ACO	待完成
Matched-Replace	完整替换	不修改	Full/F1	RQ2 的匹配替换对照	待完成
TR-RGP	残差	不修改	Full/F1	独立训练的状态转移单树	待完成
PH-RGP	不修改	预算残差	Full/F1	独立训练的信息素单树	待完成
Core-F0	残差	预算残差	Core/F0	RQ4 因子实验	待完成
Core-F1	残差	预算残差	Core/F1	RQ4 因子实验	待完成
Full-F0	残差	预算残差	Full/F0	RQ4 因子实验	待完成
Full-F1	残差	预算残差	Full/F1	本文主方法	uniform 已完成
Drop / Shuffle	训练后干预	训练后干预	固定 Full/F1 树	RQ5 联合适应分析	待完成

表 VIII
共同的 ACO 与 GP 参数

参数	数值
ACO 蚂蚁数	32
ACO 迭代数	500
候选列表大小	20
α, β	1, 2
两个残差强度	1/3, 1/3
局部搜索	关闭
GP 种群 / 代数	100/50
交叉 / 变异 / 复制	0.80/0.15/0.05
精英数 / 锦标赛大小	10/4
初始深度 / 最大深度	2-4/5
总有效节点预算	31
检查点间隔 / 保留数	5/5
每代训练实例	32
Selection / gate 实例	32/32
筛选 / 最终验证种子	1/3
每个测试实例的 ACO 种子	3
每个变体的独立 GP 运行	3
CPU 线程 / 进程	16/1
CUDA block 线程	32
搜索 / 最终评分精度	FP32 / FP64

表 IX
AS、ACS 和 MMAS 的专用参数

算法	参数	数值
AS	ρ	0.5
ACS	ρ, q_0, ξ	0.1, 0.9, 0.1
ACS	局部更新	synchronous
MMAS	ρ	0.02
MMAS	更新周期 / p_{best}	25/0.05
MMAS	分支检查周期 / 参数	100/0.05
MMAS	分支阈值	1.00001
MMAS	重启停滞上限	250

搜索使用 FP32。GPU 返回最优路线。主机使用原始 float64 距离矩阵重新计算路线长度。适应度和论文表格都使用 CPU FP64 最终评分。

B. CPU、单 GPU 与多 GPU 调度

一个 GP 运行使用一个进程和一个可见 GPU。个体层与实例层并行在 kernel 内融合。这样可以避免多个 Python 进程争用同一 GPU。多 GPU 的主要调度单位是独立 GP 运行。每个运行分配到不同设备。该 campaign 模式追求总吞吐量。

表 X
正式加速基准结果

模式	时间 (s)	路线数/s	相对 CPU16
CPU8	397.422	101,776	0.766
CPU16	304.325	132,911	1.000
GPU0	20.807	1,943,957	14.626
GPU1	20.838	1,941,110	14.605
双 GPU shard	12.776	3,165,928	23.820
Campaign [†]	21.101	3,833,840	28.9

[†] 两个工作负载开发；加速比按总吞吐量计算。

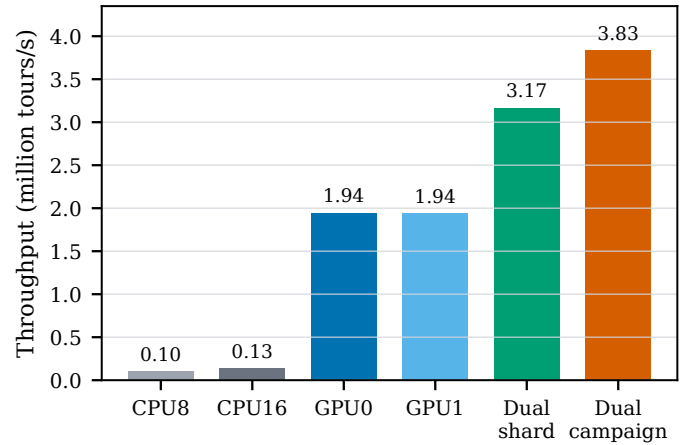


图 2. 正式加速基准的吞吐量。图内坐标保留英文。Campaign 为两个并发工作负载的总吞吐量。

双 GPU shard 模式可以拆分同一代。它降低单次运行的延迟。但是，小 shard 会降低 GPU 利用率。当有多个独立运行时，本文优先使用 campaign 模式。CPU16 是确定性回退后端，也是 FP64 审计后端。

C. 正式加速基准

正式基准使用 Intel Xeon W-2245 CPU 和两张 20-GiB NVIDIA RTX 4000 Ada Generation GPU。工作负载包含 100 个请求个体，其中 79 个行为唯一。它包含 16 个 TSP50 和 16 个 TSP100 实例。ACO 使用 ACS、32 只蚂蚁和 500 次迭代。一次工作负载构造 40,448,000 条路线。每种模式运行一次冷启动和三次热启动。表 X 报告热启动中位数。

表 XI
三个 GP 运行的实测每代时间

变体	均值 $\pm$ 标准差 (s)	运行范围 (s)
AS	34.50 $\pm$ 2.97	31.13–36.78
ACS	26.88 $\pm$ 4.43	21.97–30.57
MMAS	22.14 $\pm$ 3.34	18.31–24.41

单次运行的双 GPU 扩展率为 1.629。它低于登记的 1.7 阈值。两个独立运行的 campaign 扩展率为 1.972。它高于登记的 1.8 阈值。因此，独立任务并行更适合当前硬件。

平均 GPU 利用率约为 99%。峰值显存约为 286 MiB。约 99.8% 的热启动时间位于 CUDA kernel 内。当前负载主要受计算能力限制。继续增加缓存不太可能带来主要收益。

FP32 数值审计使用 TSP50 和 TSP100。每个规模使用 128 个实例和三个配对 ACO 种子。GPU 与 CPU 的平均参考差距之差为 -0.003445 个百分点。单侧 95% 上界为 0.000868 个百分点。该值低于预设的 0.10 个百分点阈值。

VI. 研究问题与结果分析

A. RQ1: Full-F1 与原始 ACO

表 XII 给出已完成的结果。Baseline 列是原始 ACO 的参考差距。Full-F1 列是学习方法的参考差距。 Δg 为 Full-F1 减去 baseline。负值表示 Full-F1 更好。

12 个平均差值全部小于零。12 个 bootstrap 区间也全部低于零。最小下降出现在 ACS 的 TSP50，为 0.0736 个百分点。最大下降出现在 AS 的 TSP500，为 7.8461 个百分点。Holm 校正后的 Wilcoxon 值均不大于 9.004×10^{-17} 。这些统计量描述当前数据块。它们不能替代更多独立 GP 运行。

AS 在 TSP100 上从 5.8952% 降至 4.2154%。它在 TSP500 和 TSP1000 上分别下降 7.8461 和 7.3135 个百分点。ACS 在小规模上的改进较小。它在 TSP500 上下降 3.2242 个百分点。三个 ACS 候选中有一个没有通过独立部署 gate。论文分析仍然保留该 GP 运行。删除该运行会产生选择偏差。

MMAS 在 TSP100 上从 4.1088% 降至 1.8106%。它在 TSP500 和 TSP1000 上分别下降 6.4522 和 6.6314 个百分点。当前 MMAS TSP100 的胜率为 99.2%。当前 MMAS TSP500 和 TSP1000 的胜率为 100%。

图 3 中的训练曲线有明显噪声。原因是每一代使用新的训练批量。验证曲线也不保证单调下降。因此，最后一代不一定是最佳检查点。独立的候选选择过程是必要的。

RQ1 的当前答案：在全部 12 个已完成的先导比较中，Full-F1 的平均参考差距都低于对应的原始 ACO。因此，当前结果支持完整方法有效。该结果不能说明改进一定来自 residual、双树或 Full-F1 配置。

B. RQ2: 残差与完整替换

待完成。主要比较是 TR-RGP 与 Matched-Replace。两者都使用一棵状态转移树、Full 终端、F1 函数和相同节点预算。唯一差异是树输出的接入方式。TR-RGP 修正原始分数。Matched-Replace 直接生成完整分数。

Legacy-GP 是辅助基线。它同时改变终端和函数。因此，Legacy-GP 不能单独识别 residual 的作用。

RQ2 的当前答案：目前不能说明 residual 比完整替换更好。Full-F1 优于 baseline ACO 不能回答这个问题。

C. RQ3: 双树与单树

待完成。主要比较包含 Full-F1、独立训练的 TR-RGP 和独立训练的 PH-RGP。三者共享 31 个总有效节点。双树的 31 个节点在两棵树之间分配。它不是每棵树各 31 个节点。这种设置控制了总语法容量。

31 与 62 节点的容量实验用于检查结果是否依赖预算。训练后删除一棵树只能分析已有个体的机制。它不能替代独立训练的单树对照。

RQ3 的当前答案：目前不能说明双树优于单树。

D. RQ4: 终端与函数配置

待完成。因子实验包含 Core-F0、Core-F1、Full-F0 和 Full-F1。Full 与 Core 的平均差异表示终端主效应。F1 与 F0 的平均差异表示函数主效应。两种差异是否相互依赖由交互效应判断。

Core 保留四个状态转移终端和四个信息素终端。Full 增加进度、熵、停滞和蚁群频率信息。F1 增加 MIN、MAX 和 ABS。该实验可以识别没有必要的输入或函数。

RQ4 的当前答案：现有终端和函数在类型和数值上是有效的。它们是否存在过度设计仍需实验回答。

E. RQ5: 联合适应

待完成。第一项干预将状态转移树替换为零。第二项干预将信息素树替换为零。第三项干预在独立 GP 运行之间打乱两棵树的配对。若打乱后性能下降，则说明存在配对相关的联合适应。该结果仍不能代替 RQ3 的独立单树训练。

RQ5 的当前答案：目前没有结论。

F. RQ6: 泛化能力

当前结果支持 uniform 欧氏 TSP 上的规模迁移。训练和验证只使用 TSP100。TSP50、TSP500 和 TSP1000 都是未见规模。三个 ACO 变体在三个未见规模上的平均 Δg 都小于零。在 TSP500 和 TSP1000 上，绝对下降通常更大。

该结果不能证明分布迁移。Cluster TSP500、Gaussian TSP500 和 TSPLIB 的结果仍待完成。

RQ6 的当前答案：先导结果支持相同分布下的跨规模迁移。分布外迁移仍无结论。

G. RQ7: 计算效率

在登记的工作负载上，单 GPU 相对 CPU16 加速 14.626 倍。双 GPU shard 相对 CPU16 加速 23.820 倍。它相对单 GPU 的扩展率为 1.629。两个独立工作负载并行时，扩展率为 1.972。

RQ7 的当前答案：“个体 $\times$ 实例”融合使正式 ACO 预算下的 GP 训练可行。多 GPU 环境应优先并行独立 GP 运行。这些数值只适用于登记的 RTX 4000 Ada 工作负载。其他硬件需要重新测量。

VII. 局限性

当前主要局限是 GP 运行数。三个运行可以发现较大的失败。它们不能充分估计进化随机性。正式实验需要 30 个独立 GP 运行。

训练只使用 uniform TSP100。当前结论只适用于报告的 AS、同步 ACS 和 MMAS 参数。本文没有启用局部搜索。分布迁移实验尚未完成。

表 XII
已完成的 UNIFORM 测试结果。区间是 Δg 的分层 BOOTSTRAP 95% 区间。胜/平/负以百分比报告。

ACO	n	Baseline gap (%)	Full-F1 gap (%)	Δg (百分点)	95% 区间	胜/平/负
AS	50	1.8803	1.7479	-0.1324	[-0.2076, -0.0718]	54.7/2.8/42.4
AS	100	5.8952	4.2154	-1.6799	[-1.9680, -1.4739]	92.1/0.0/7.9
AS	500	22.9874	15.1413	-7.8461	[-8.3979, -7.3727]	100.0/0.0/0.0
AS	1000	30.4224	23.1088	-7.3135	[-8.5940, -6.1418]	100.0/0.0/0.0
ACS	50	0.7326	0.6589	-0.0736	[-0.1050, -0.0432]	46.6/17.6/35.9
ACS	100	1.9331	1.6412	-0.2918	[-0.3954, -0.2091]	64.7/0.0/35.3
ACS	500	16.1641	12.9399	-3.2242	[-4.5698, -1.3932]	93.8/0.0/6.2
ACS	1000	21.6293	19.2041	-2.4252	[-3.3961, -0.8317]	94.8/0.0/5.2
MMAS	50	0.5941	0.4727	-0.1215	[-0.1549, -0.0820]	52.3/22.4/25.2
MMAS	100	4.1088	1.8106	-2.2982	[-2.4240, -2.1223]	99.2/0.0/0.8
MMAS	500	20.6291	14.1769	-6.4522	[-8.1652, -4.7067]	100.0/0.0/0.0
MMAS	1000	25.4885	18.8571	-6.6314	[-8.3746, -4.8120]	100.0/0.0/0.0

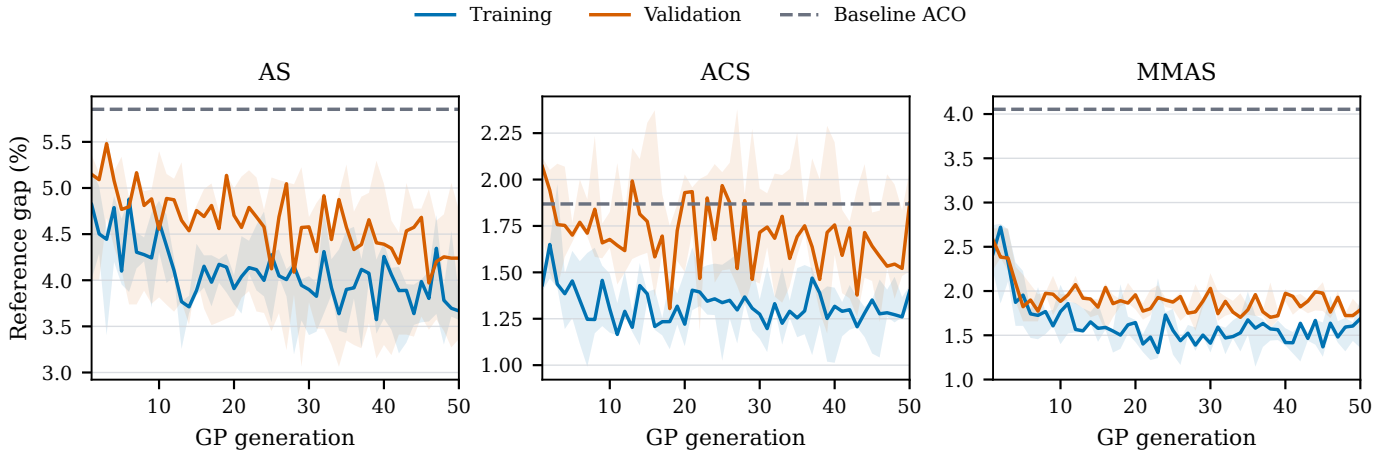


图 3. TSP100 训练与验证曲线。曲线为三个独立 GP 运行的中位数。阴影为最小值和最大值。每代使用新的训练批量。图内坐标保留英文。

参考路线均有效，但不都带有全局最优性证书。因此，本文谨慎使用“参考差距”。由于所有方法使用同一参考路线，该指标仍适合配对比较。

GPU 搜索使用 FP32。CPU 与 GPU 不保证产生相同的随机轨迹。最终路线长度使用 FP64 计算。登记的数值非劣审计已经通过。该结果不表示逐位一致。

RQ1 结果不能提供因果解释。Residual 与替换、双树与单树、函数与终端配置、联合适应都需要各自的受控实验。

VIII. 结论

本文提出 RMTGP-ACO。方法使用两棵强类型残差树。状态转移树对原始分数进行有界修正。信息素树在原始增强预算内重新分配增量。两个接口都能通过零输出严格恢复 baseline ACO。两棵树共享 31 个总有效节点。

当前先导实验对 AS、ACS 和 MMAS 各进行三个独立 GP 运行。Full-F1 在全部 12 个 uniform 测试组合中取得更低的平均参考差距。该结果覆盖 TSP50 至 TSP1000。融合 CUDA 评估器显著降低了 GP-ACO 嵌套计算成本。

当前证据支持继续开展正式实验。它还不能证明 residual 优于完整替换。它也不能证明双树优于单树。登记的消融实验和容量实验是得到这些结论的必要条件。

参考文献

- [1] M. Dorigo, V. Maniezzo, and A. Coloni, “Ant system: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [2] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004.
- [3] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA, USA: MIT Press, 1992.
- [4] R. Poli, W. B. Langdon, and N. F. McPhee, *A Field Guide to Genetic Programming*. Lulu Enterprises, 2008, with contributions by J. R. Koza.
- [5] B.-C. Lin, Y. Mei, and M. Zhang, “Automated design of state transition rules in ant colony optimization by genetic programming: A comprehensive investigation,” *Memetic Computing*, vol. 17, p. 2, 2025.
- [6] T. Stützle, “ACOTSP, version 1.0,” Software package, 2004, version 1.03 used as the algorithmic reference.
- [7] D. J. Montana, “Strongly typed genetic programming,” *Evolutionary Computation*, vol. 3, no. 2, pp. 199–230, 1995.
- [8] F.-A. Fortin, F.-M. D. Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, “DEAP: Evolutionary algorithms made easy,” *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.
- [9] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [10] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [11] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman and Hall/CRC, 1994.
- [12] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

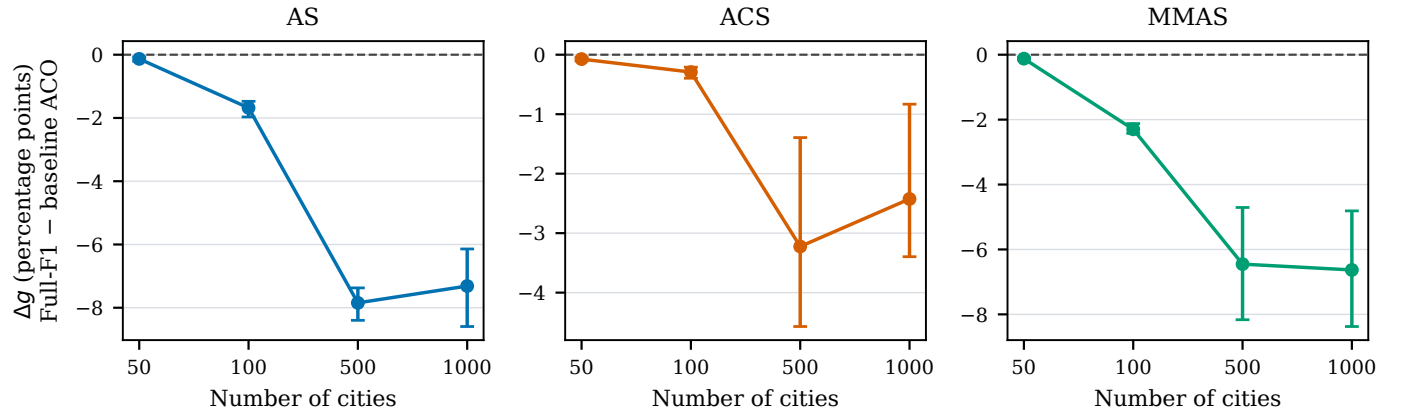


图 4. Full-F1 与 baseline ACO 的参考差距之差。误差线为分层 bootstrap 95% 区间。负值支持 Full-F1。图内坐标保留英文。

- [13] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. ACM, 2015, pp. 1–6.
- [14] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.
- [15] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in *Proceedings of the Workshop on Machine Learning Systems at NeurIPS*, 2017.